

CNN Architecture Design \& Advanced Operations

A comprehensive deep dive into hierarchical feature learning, strided and dilated convolutions, multi-channel processing, parameter counting, and ResNet/Bottleneck design patterns.

Session 8A \textperiodcentered\ Read alongside the lecture slides \textperiodcentered\ Every slide example worked out in full

\smallskip \noindent\small\color{mutedtext}% Prepared for the Work Integrated Learning Programmes (WILP), BITS Pilani.} \smallskip \noindent**How to use this companion.** This guide extends basic CNN principles into advanced convolutional operations, exact parameter counting, multi-scale feature extraction, and modern architectural design patterns (VGG, ResNet, Inception, EfficientNet). Every formula is broken down step by step, and all numerical examples from the lecture deck are solved in full. \colorbox{intuFrame!10}{\textcolor{intuFrame}{\textbf{Blue}}}} boxes explain intuition; \colorbox{workFrame!10}{\textcolor{workFrame}{\textbf{Green}}}} boxes present fully solved numerical problems; \colorbox{watchFrame!10}{\textcolor{watchFrame}{\textbf{Red}}}} boxes highlight design pitfalls; \colorbox{keyFrame!10}{\textcolor{keyFrame}{\textbf{Purple}}}} boxes summarize core takeaways. \linewidth\medskip

Hierarchical feature learning and CNN layer breakdown

How deep networks build high-level concepts from low-level visual primitives.

Convolutional Neural Networks (CNNs) process images through a sequence of hierarchical transformations:

$$\text{INPUT} \rightarrow [\text{CONV} \rightarrow \text{ACTIVATION} \rightarrow \text{POOL}] \times N \rightarrow \text{FC} \rightarrow \text{OUTPUT}$$

As information flows through the network: \begin{enumerate}[\leftmargin=1.5em]

Early Layers (Layers 1--2): Detect simple, low-level features such as vertical/horizontal edges, color gradients, and corners. Filters act like localized Gabor filters.

Middle Layers (Layers 3--4): Combine low-level primitives into mid-level shapes, textures, and repeating patterns (e.g., circles, grilles, wheels).

Deep Layers (Layers 5+): Assemble shapes into high-level object parts and full semantic concepts (e.g., dog faces, car bodies, text characters). \end{enumerate}

📖 The intuition

Hierarchical feature learning is like reading a book. First you recognize individual stroke angles (edges), combine them into letters (simple shapes), assemble letters into words (object parts), and finally grasp the full sentence meaning (complete object classification).

✓ Key takeaway

CNNs learn visual hierarchies automatically from data: early layers capture spatial edges, middle layers group shapes, and deep layers identify complex semantic objects.

Advanced convolution operations: Stride, padding, and dilated convolutions

Control spatial dimensions and expand receptive fields without inflating parameter counts.

1. Padding: Without padding, convolving an $N \times N$ image with a $K \times K$ kernel shrinks the output to $(N - K + 1) \times (N - K + 1)$, stripping border information rapidly. To preserve input dimensions (**Same Padding**), we pad the border with zeros. For kernel size K and stride $S = 1$:

$$P = \frac{K - 1}{2}$$

2. Stride: Stride S is the step size the filter takes as it slides across the image. Striving $S > 1$ downsamples the spatial resolution directly during convolution. **General Output Dimension Formula:**

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1$$

3. Dilated (Atrous) Convolutions: Dilated convolution inserts spaces (holes) between kernel elements according to a **dilation rate r** .

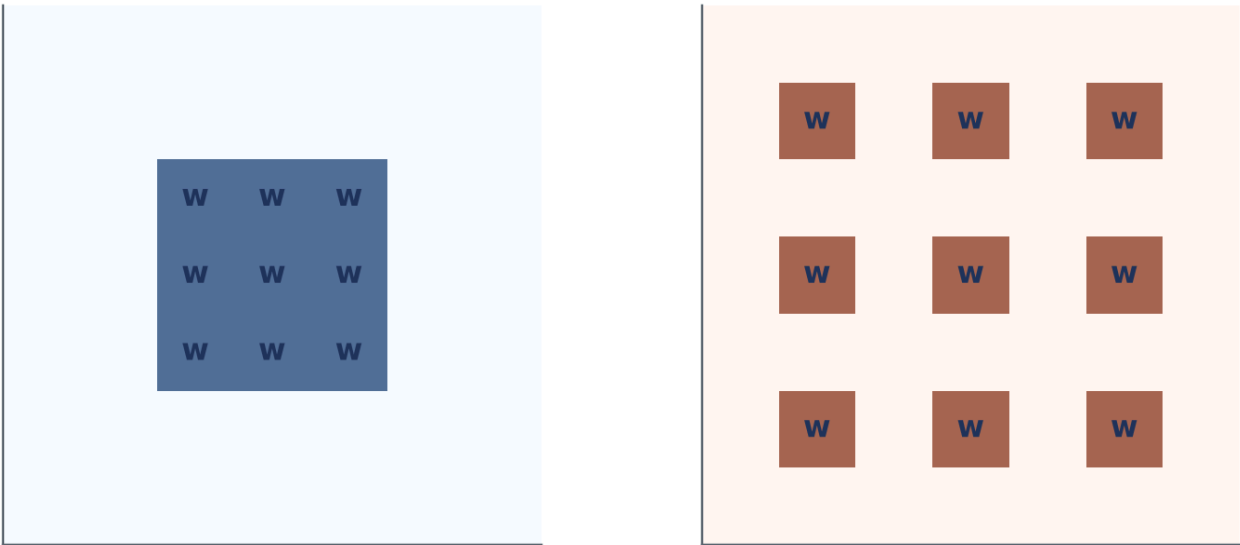
For rate $r = 1$: Standard convolution (no spaces).

For rate $r = 2$: Inserts 1 zero between adjacent filter weights.

Effective Kernel Size: $K' = K + (K - 1)(r - 1)$. Dilated convolutions expand the network's receptive field exponentially without losing spatial resolution or adding learnable parameters!

Standard Conv (Rate $r=1$, 3×3)

Dilated Conv (Rate $r=2$, Field 5×5)



Standard Convolution ($r=1$, 3×3) vs Dilated Convolution ($r=2$, 5×5 effective field) with identical 9 parameters.

🔑 Worked example: Solving slide problem 29: Convolution, Channels, and Max-Pooling

Let us solve the exact multi-part problem from Slide 29:

1. **Part (a):** Given an 8×8 input matrix, kernel size 3×3 , stride $S = 1$, no padding ($P = 0$). Determine output shape.

$$O = \left\lfloor \frac{8+2(0)-3}{1} \right\rfloor + 1 = \frac{5}{1} + 1 = 6 \implies 6 \times 6.$$

2. **Part (b):** If 5 such filters are applied, what is the output shape?

Applying 5 filters creates 5 output feature channels. Output shape: $6 \times 6 \times 5$.

3. **Part (c):** Apply a 4×4 max-pooling filter with stride $S = 2$ to the output of Part (b). What is the final shape?

$$\text{Spatial output: } O_{\text{pool}} = \left\lfloor \frac{6-4}{2} \right\rfloor + 1 = \frac{2}{2} + 1 = 2 \implies 2 \times 2.$$

Pooling preserves the channel depth of 5.

Final Output Shape: $2 \times 2 \times 5$.

Pooling layers: Spatial reduction with zero parameters

Downsample feature maps while guaranteeing zero added weights or biases.

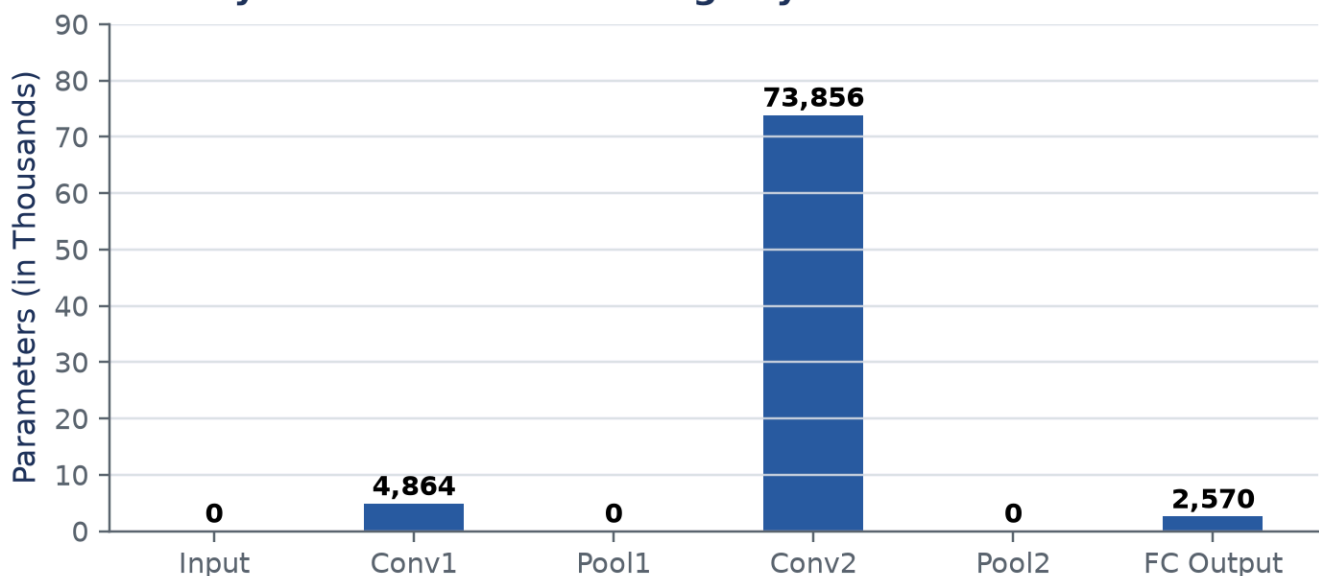
Pooling layers perform spatial downsampling across height and width while operating independently on each channel. \begin{itemize}[\leftmargin=1.5em]

Max Pooling: Extracts the maximum value in each $k \times k$ region. Retains the strongest feature activations and grants local translation invariance.

Average Pooling: Computes the arithmetic mean over each region. Provides smoother downsampling.

\end{itemize}

CNN Layer Parameters: Pooling Layers Have ZERO Parameters



Parameter counts across CNN layers: Pooling layers contribute exactly ZERO learnable parameters.

⚠ Watch out

Remember: Pooling layers contain NO learnable weights or biases! Their parameters are fixed mathematical functions (max or mean). They possess hyperparameters (k, S) , but zero parameters.

✓ Key takeaway

Pooling reduces spatial dimensions $(H, W) \rightarrow (H', W')$ and lowers memory usage while preserving feature depth C with zero trainable parameters.

Fully Connected (FC) layers vs Global Average Pooling

Transitioning spatial feature maps into class classification scores.

In a traditional CNN, feature maps $(H \times W \times C)$ are flattened into a 1D vector of length $H \cdot W \cdot C$ and passed through one or more Fully Connected (FC) layers:

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

For input dimension n and output neurons m , parameter count is $m(n + 1)$.

Disadvantages of FC layers:

- Extreme parameter consumption (often $> 80\%$ of total network weights).

- High risk of overfitting.

- Completely destroys spatial relationships.

Modern Alternative: Global Average Pooling (GAP): Instead of flattening, GAP averages each $H \times W$ feature map into a single scalar value, converting $(H \times W \times C) \rightarrow (1 \times 1 \times C)$. This eliminates millions of FC parameters while enforcing direct correspondence between feature maps and categories.

✓ Key takeaway

Modern architectures replace parameter-heavy FC layers with Global Average Pooling (GAP) and 1×1 convolutions to prevent overfitting and support flexible input resolutions.

Architectural design patterns: ResNet, Bottlenecks, and Inception

Master key modular patterns used in state-of-the-art vision models.

Modern deep CNN architectures (ResNet, Inception, MobileNet) rely on five fundamental design patterns:

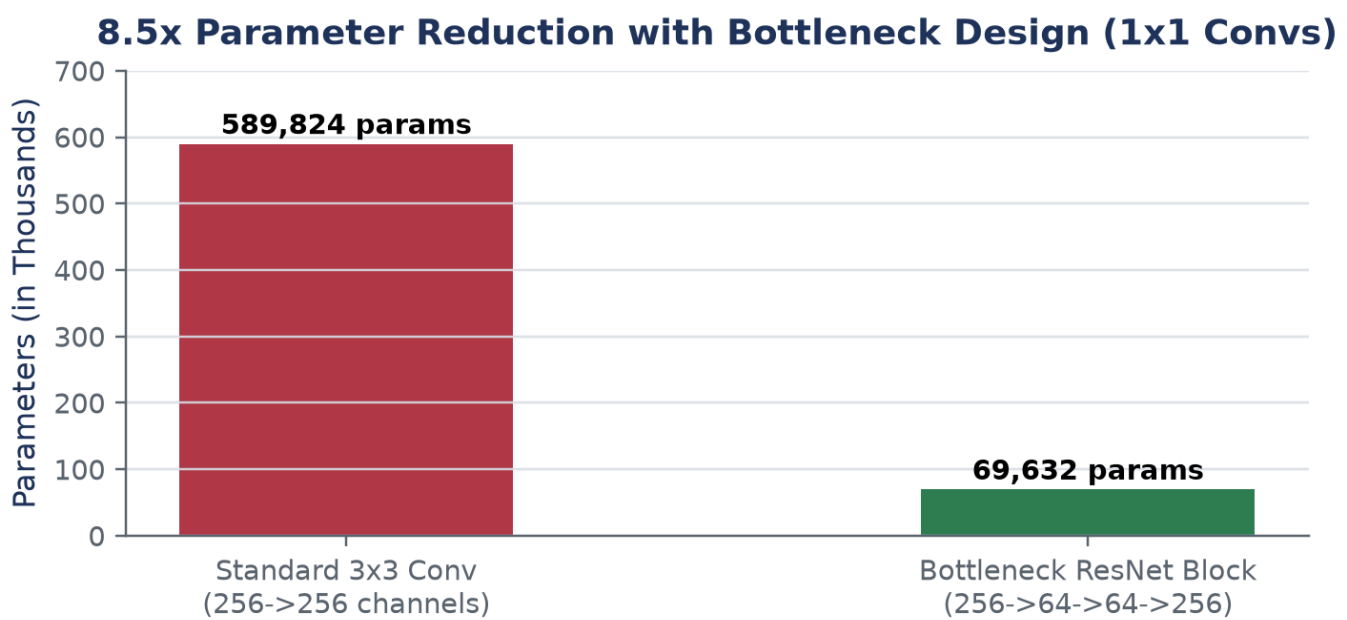
- Spatial Reduction + Channel Expansion:** Halving spatial resolution $(H/2, W/2)$ while doubling channel depth $(2C)$ maintains total representation capacity.

- Repetitive Modular Blocks:** Stacking identical modular building blocks (like VGG 3×3 blocks or ResNet residual blocks) simplifies network scaling.

- Skip/Residual Connections:** Adds identity shortcuts $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$. Gradients flow directly through identity paths during backpropagation, solving the vanishing gradient problem and enabling 100+ layer networks.

- Bottleneck Design:** Uses 1×1 convolutions to squeeze channels before expensive 3×3 convolutions, then expands channels back. Reduces parameter counts by over $8\times$.

Multi-Scale Processing (Inception): Applies 1×1 , 3×3 , and 5×5 filters in parallel to capture multi-scale spatial features simultaneously.



Parameter comparison: Standard 3×3 Conv vs Bottleneck ResNet Block achieving an 8.5x reduction in weights.

🔗 Worked example: Bottleneck Block Parameter Reduction Calculation

Compare parameters for processing a 256-channel feature map:

- Standard 3×3 Convolution (256 in, 256 out):**
Params = $(3 \times 3 \times 256 + 1) \times 256 \approx 3 \times 3 \times 256 \times 256 = 589,824$ parameters.
- Bottleneck Block (1×1 reduce to 64 $\rightarrow 3 \times 3$ conv $\rightarrow 1 \times 1$ expand to 256):**
 Layer 1 (1×1 conv, 256 $\rightarrow$ 64): $1 \times 1 \times 256 \times 64 = 16,384$.
 Layer 2 (3×3 conv, 64 $\rightarrow$ 64): $3 \times 3 \times 64 \times 64 = 36,864$.
 Layer 3 (1×1 conv, 64 $\rightarrow$ 256): $1 \times 1 \times 64 \times 256 = 16,384$.
Total Bottleneck Params = $16,384 + 36,864 + 16,384 = 69,632$ parameters.
- Compute Reduction Factor:**
 $\frac{589,824}{69,632} \approx 8.47\times$ fewer parameters!

Trade-offs: Depth vs. Width and Design Framework

How to systematically choose model depth, width, and resolution.

Depth vs. Width Guidelines:	Aspect & Deeper (More Layers) & Wider (More Channels)
Feature Complexity	High hierarchical features & Moderate features
Receptive Field	Expands with depth & Unchanged
Training Difficulty	Harder (requires skip connections) & Easier gradient flow
Inference Speed	Slower (sequential operations) & Faster (parallel computing)
Systematic Design Decision Framework:	
Start with a proven baseline: Use pretrained ResNet-50 or EfficientNet via transfer learning.	

Estimate constraints: Small dataset ($< 10K$ samples) $\implies$ shallow network or fine-tuning to prevent overfitting. Large dataset ($> 1M$ samples) $\implies$ deep model (50--152 layers).

Apply compound scaling (EfficientNet): Scale depth, width, and image resolution jointly using a fixed ratio for optimal efficiency.

✓ Key takeaway

Residual connections permit extreme depth (100+ layers), bottleneck blocks keep compute lean, and compound scaling balances depth, width, and resolution.

Glossary & symbols

	Key terms.
[leftmargin=9.5em,style=nextline,font=\normalfont\bfseries\color{navy}]	
[Same Padding]	Zeros added around image borders so output height/width match input dimensions.
[Stride]	Step size in pixels taken by a convolution filter as it slides over the input grid.
[Dilated Conv]	Convolution inserting spaces between filter weights to expand receptive field.
[Global Average Pooling]	Averaging entire spatial feature maps to scalars, replacing FC layers.
[Residual Connection]	Identity shortcut connection $y = F(x) + x$ enabling stable gradient flow in deep networks.
[Bottleneck Block]	$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ convolution structure reducing channel dimensionality and parameter cost.
	Symbols at a glance.
	Symbol & Meaning
	N, K, P, S & Input size, Kernel size, Padding size, and Stride
	r & Dilation rate for dilated convolutions
	C_{in}, C_{out} & Number of input channels and output filters
	$F(x) + x$ & Residual mapping formulation